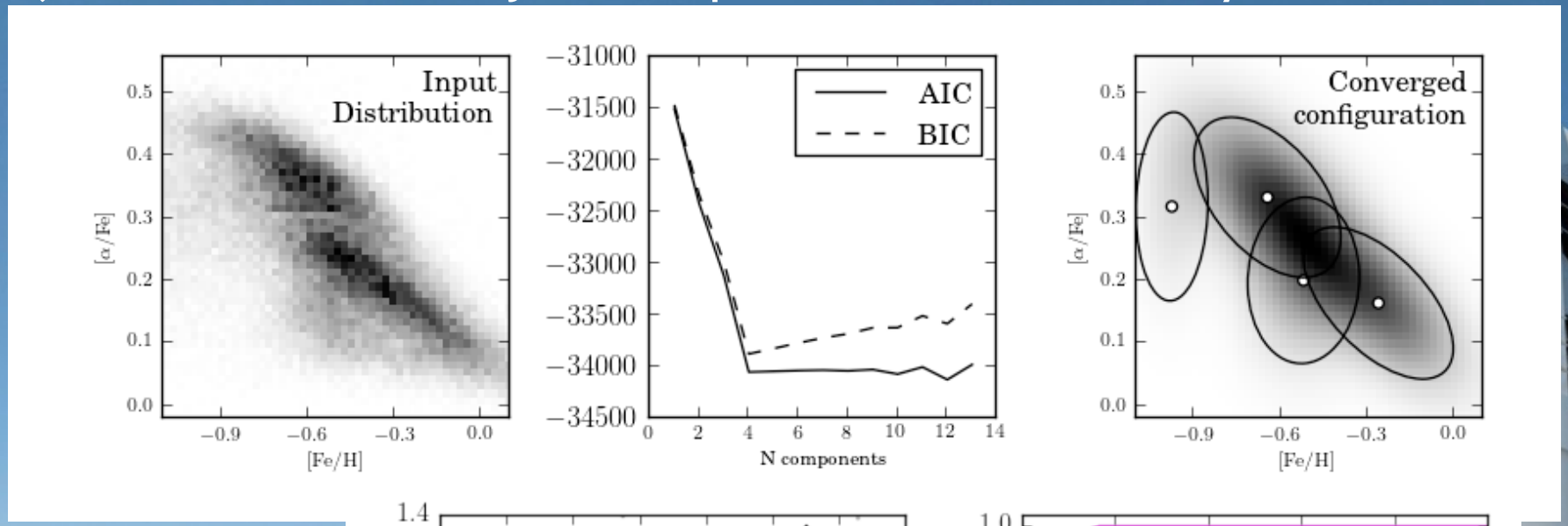
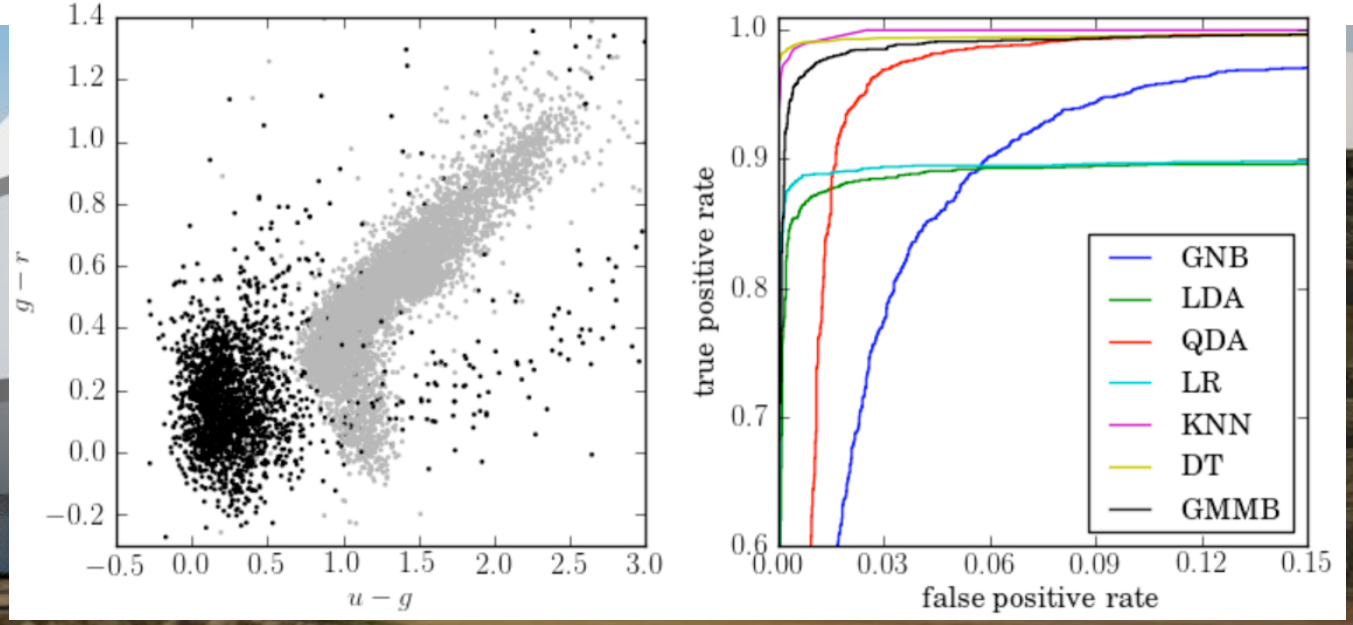
Week 10: (supervised) Classification

Željko Ivezić and Mario Jurić, Department of Astronomy, UW





Outline

- Clustering (unsupervised classification)
  – 1-D hypothesis testing
  – clustering with Gaussian Mixture models (GMM)
  – hierarchical clustering algorithm
- (supervised) Classification
  – potpourri of supervised classification methods: naive Bayes, quadratic discriminant analysis, GMM, KNN, support vector machines
  – classification comparison with ROC curves

Clustering (unsupervised classification)

Here “unsupervised” means that there is no prior information about the number and properties of clusters.

“Clustering” in astronomy refers to a number of different aspects of data analysis.

Given a multivariate point data set, we can ask whether it displays any structure, that is, concentrations of points. Alternatively, when a density estimate is available we can search for “overdensities.”

Another way to interpret clustering is to seek a partitioning or segmentation of data into smaller parts according to some criteria.

Supervised classification

Here “supervised” means that there is prior information about the number and properties of clusters: for a training sample, we know the so-called “class labels” (for each data point in the training sample, we know to which cluster it belongs; characterizing these known clusters is typically easier than finding and characterizing unknown clusters)

There are two basic types of classification methods: **generative classification** methods model the underlying density field (i.e. it relies on density estimation methods), and **discriminative classification** methods which focus on finding the decision boundary which separates classes directly. The former are easier to interpret, the latter often work better in high-D cases.

Generative classification

Given a set of data $\{\mathbf{x}\}$ consisting of N points in D dimensions, such that x_i^j is the jth feature of the ith point, and a set of discrete labels $\{y\}$ drawn from K classes, with values y_k, Bayes' theorem describes the relation between the labels and features:

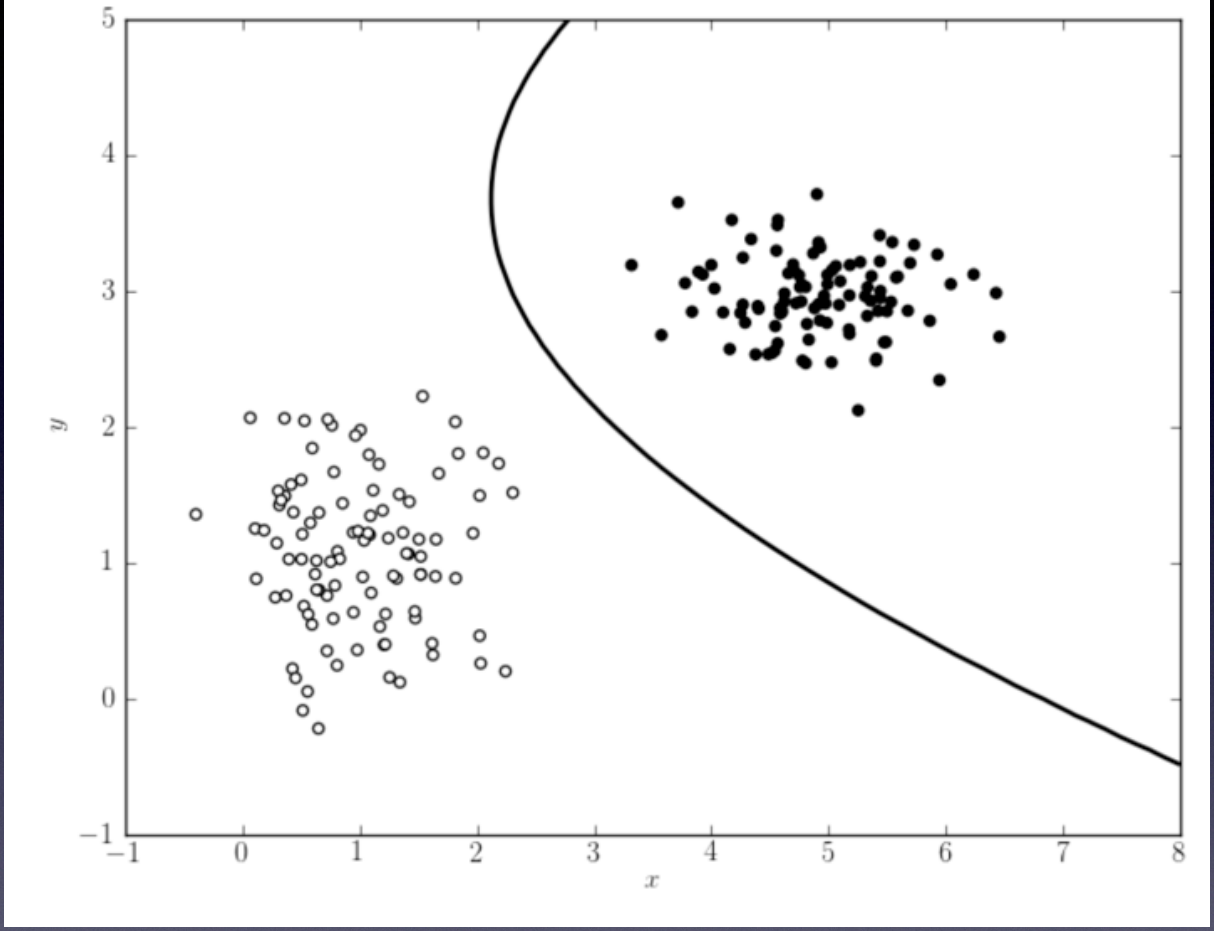
$$p(y_k|\mathbf{x}_i) = \frac{p(\mathbf{x}_i|y_k)p(y_k)}{\sum_i p(\mathbf{x}_i|y_k)p(y_k)}. \tag{9.6}$$

If we knew the full probability densities $p(\mathbf{x}, y)$ it would be straightforward to estimate the classification likelihoods directly from the data. If we chose not to fully sample $p(\mathbf{x}, y)$ with our training set we can still define the classifications by drawing from $p(y|\mathbf{x})$ and comparing the likelihood ratios between classes (in this way we can focus our labeling on the specific, and rare, classes of source rather than taking a brute-force random sample).

In generative classifiers we are modeling the class-conditional densities explicitly, which we can write as $p_k(\mathbf{x})$ for $p(\mathbf{x}|y = y_k)$, where the class variable is, say, $y_k = 0$ or $y_k = 1$. The quantity $p(y = y_k)$, or π_k for short, is the probability of any point having class k, regardless of which point

The task of learning the best classifier then becomes the task of estimating the p_k's. This approach means we will be doing multiple separate *density estimates* using many of the techniques introduced

Discriminative classification



$$\text{completeness} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}},$$

$$\text{contamination} = \frac{\text{false positives}}{\text{true positives} + \text{false positives}}.$$

In this simple example, perfect separation is possible; generally not the case!

Discriminative classification

discriminant function

$$g(\mathbf{x}) = f(y|\mathbf{x}) = \int y\, p(y|\mathbf{x})\, dy \tag{9.7}$$

$$= 1 \cdot p(y=1|\mathbf{x}) + 0 \cdot p(y=0|\mathbf{x}) = p(y=1|\mathbf{x}). \tag{9.8}$$

If we now apply Bayes' rule (eq. 3.10), we find (cf. eq. 9.6)

$$g(\mathbf{x}) = \frac{p(\mathbf{x}|y=1)\, p(y=1)}{p(\mathbf{x}|y=1)\, p(y=1) + p(\mathbf{x}|y=0)\, p(y=0)} \tag{9.9}$$

$$= \frac{\pi_1 p_1(\mathbf{x})}{\pi_1 p_1(\mathbf{x}) + \pi_0 p_0(\mathbf{x})}. \tag{9.10}$$

Bayes classifier

Making the discriminant function yield a binary prediction gives the abstract template called a *Bayes classifier*. It can be formulated as

$$\widehat{y} = \begin{cases} 1 & \text{if } g(\mathbf{x}) > 1/2, \\ 0 & \text{otherwise,} \end{cases} \tag{9.11}$$

$$= \begin{cases} 1 & \text{if } p(y=1|\mathbf{x}) > p(y=0|\mathbf{x}), \\ 0 & \text{otherwise,} \end{cases} \tag{9.12}$$

$$= \begin{cases} 1 & \text{if } \pi_1 p_1(\mathbf{x}) > \pi_0 p_0(\mathbf{x}), \\ 0 & \text{otherwise.} \end{cases} \tag{9.13}$$

This is easily generalized to any number of classes K, since we can think of a $g_k(\mathbf{x})$ for each class (in a two-class problem it is sufficient to consider $g(\mathbf{x}) = g_1(\mathbf{x})$). The Bayes classifier is a template in the sense that one can plug in different types of model for the p_k's and the π's.

Decision boundary

The *decision boundary* between two classes is the set of x values at which each class is equally likely; that is,

$$\pi_1 p_1(\mathbf{x}) = \pi_2 p_2(\mathbf{x}); \tag{9.14}$$

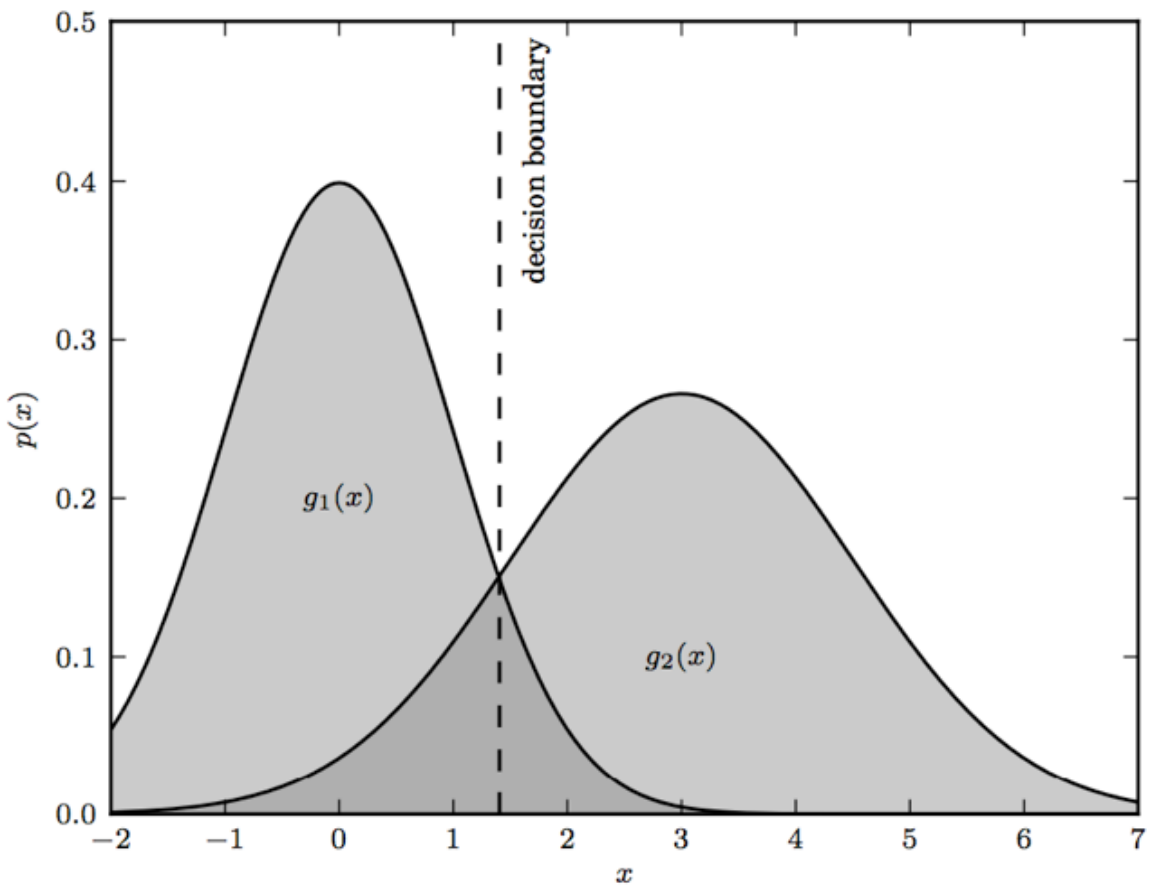
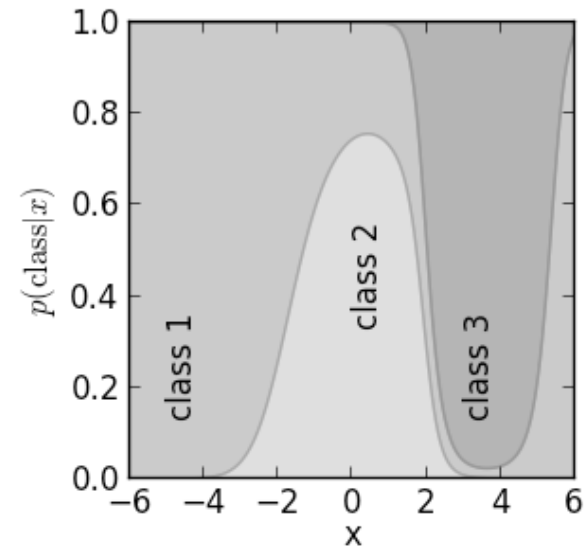


Figure 9.1.: An illustration of a decision boundary between two Gaussian distributions.



Naive Bayes classifier

The Bayes classifier formalism presented above is conceptually simple, but can be very difficult to compute: in practice, the data $\{\mathbf{x}\}$ above may be in many dimensions, and have complicated probability distributions. We can dramatically reduce the complexity of the problem by making the assumption that all of the attributes we measure are conditionally independent. This means that

$$p(x^i, x^j | y_k) = p(x^i | y) p(x^j | y_k), \tag{9.15}$$

$$p(y_k | x^0, x^1, \ldots, x^N) = \frac{\prod_i p(x^i | y_k) p(y_k)}{\sum_j \prod_i p(x^i | y_j) p(y_j)}.$$

prior for a given class!

recall:

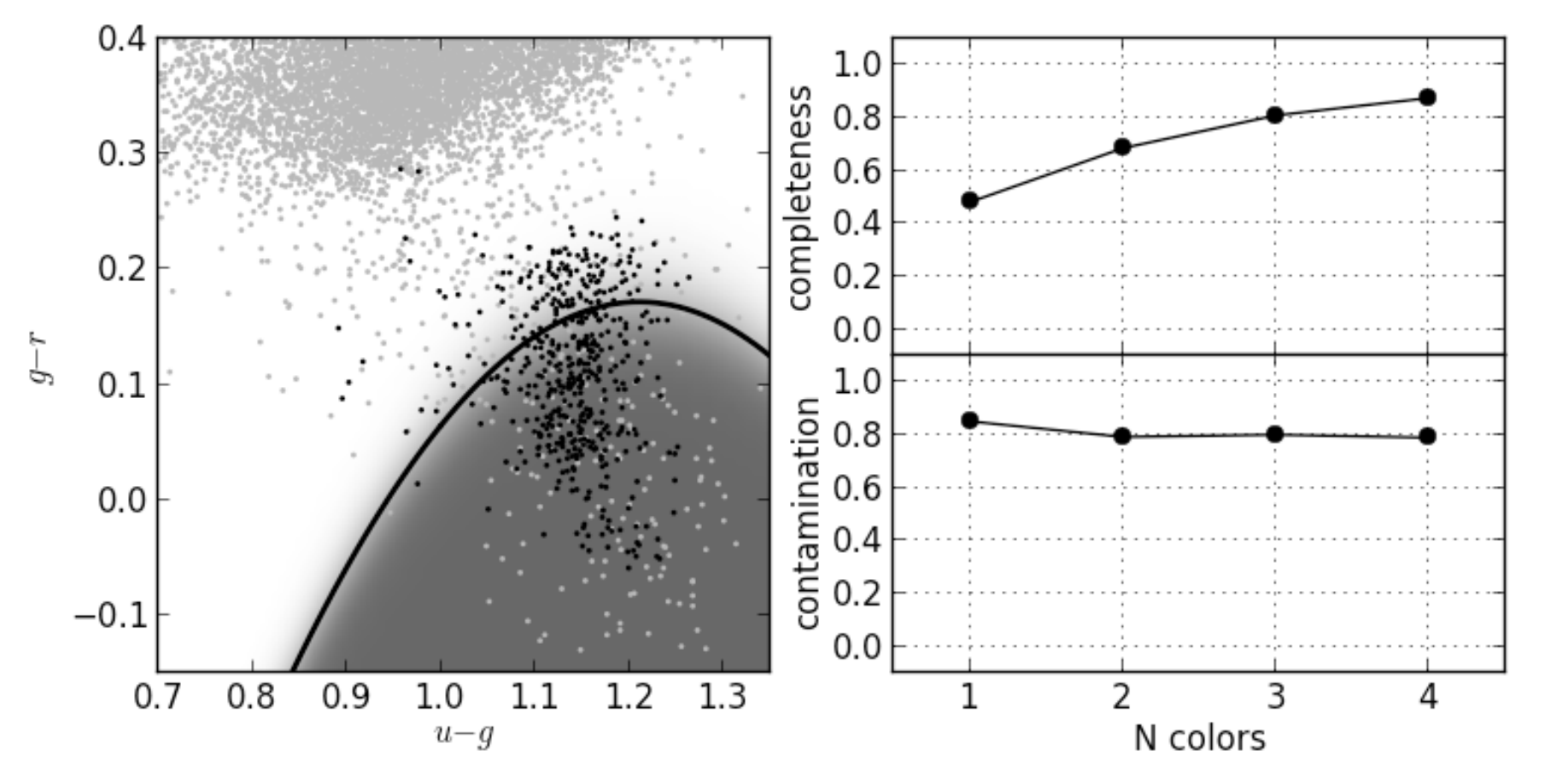
$$p(j | x_i) = \frac{\alpha_j \mathcal{N}(\mu_j, \sigma_j)}{\sum_{j=1}^{M} \alpha_j \mathcal{N}(\mu_j, \sigma_j)}.$$

Gaussian naive Bayes classifier assumes that the data can be modeled by a sum of axis-aligned multi-variate gaussians

Gaussian naive Bayes classification of RR Lyrae using colors

- make this plot by running
  %run fig_rrlyrae_naivebayes.py

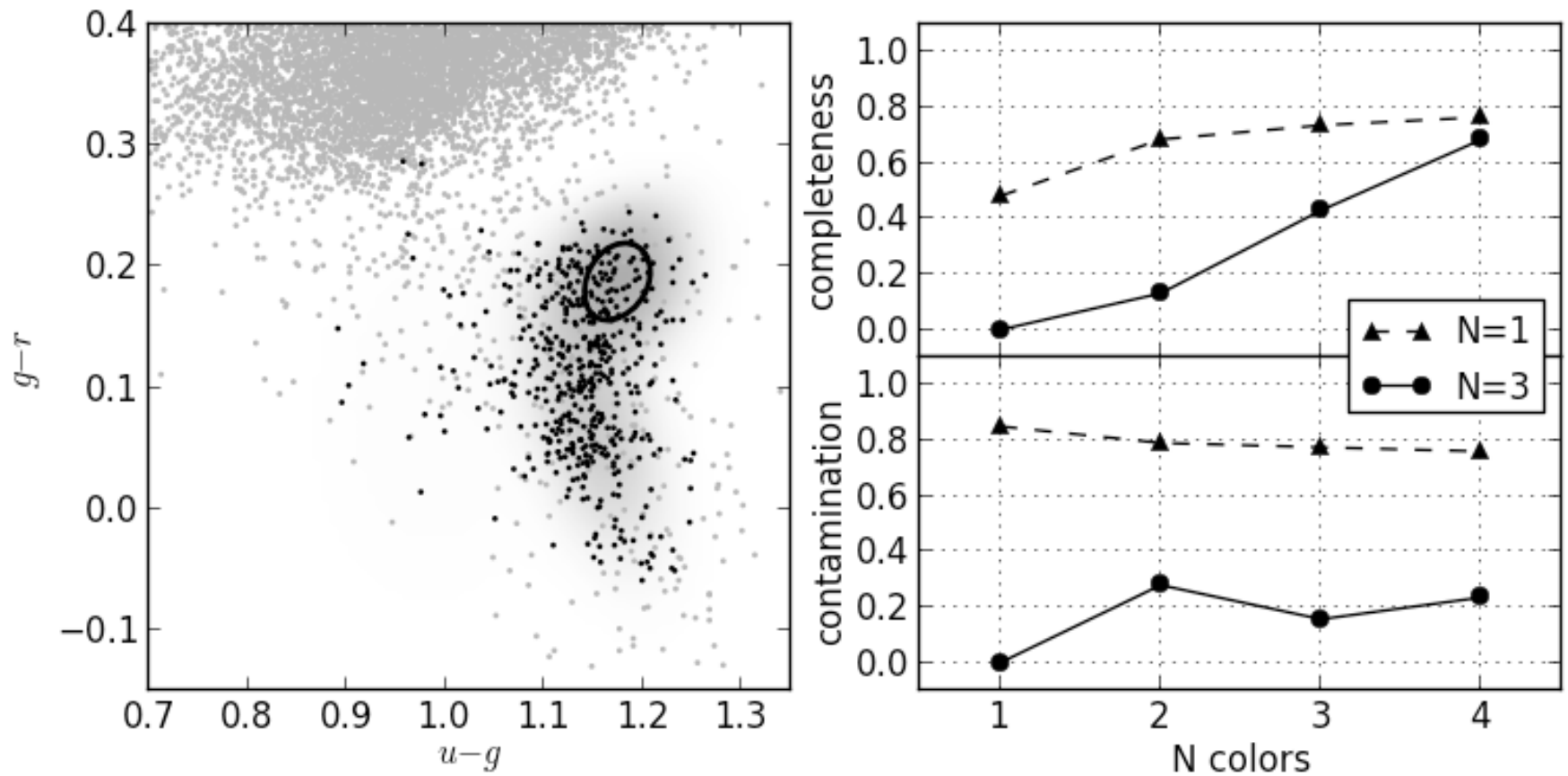
Black dots are 483 RR Lyrae, gray dots are the much larger sample of ~93,000 other stars (for this real-world dataset it is impossible to achieve perfect classification!)



Gaussian Mixture Bayes classification of RR Lyrae

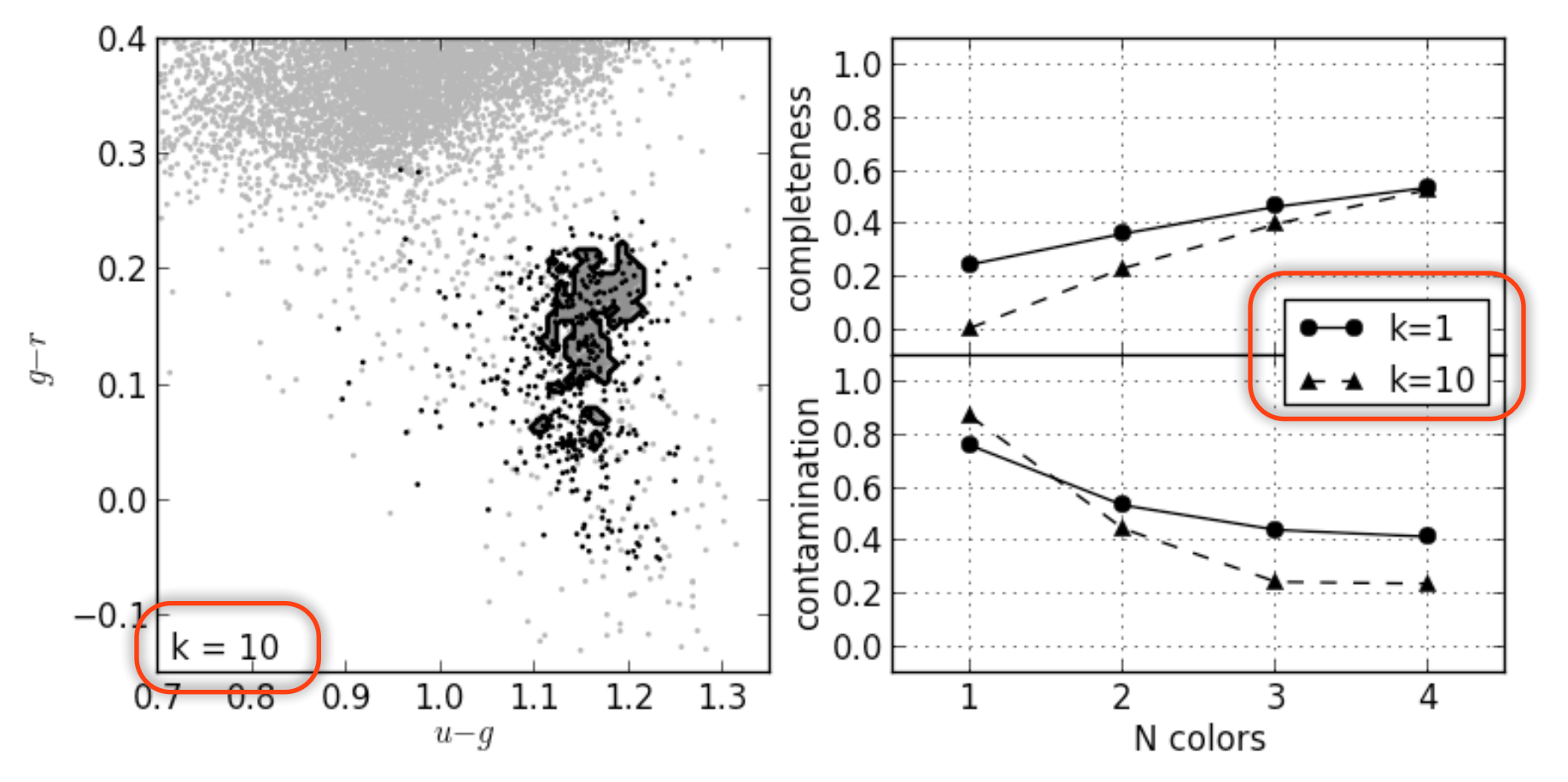
- make this plot by running
  %run fig_rrlyrae_GMMbayes.py

GMM Bayes classifier does **not** assume that gaussian components are aligned with coordinate axes



K nearest neighbors classification of RR Lyrae

- make this plot by running
  %run fig_rrlyrae_knn.py



KNN classifier is a non-parametric classifier: labels of K neighbors are weighted by distance (need large samples)

Discriminative classification

With nearest-neighbor classifiers we started to see a subtle transition—while clearly related to Bayes classifiers using variable-bandwidth kernel estimators, the class density estimates were skipped in favor of a simple classification decision. This is an example of *discriminative classification*, where we directly model the decision boundary between two or more classes of source. Recall, for $y \in \{0, 1\}$, the discriminant function is given by $g(x) = p(y = 1|x)$. Once we have it, no matter how we obtain it, we can use the rule

$$\widehat{y} = \begin{cases} 1 & \text{if } g(x) > 1/2, \\ 0 & \text{otherwise,} \end{cases} \tag{9.28}$$

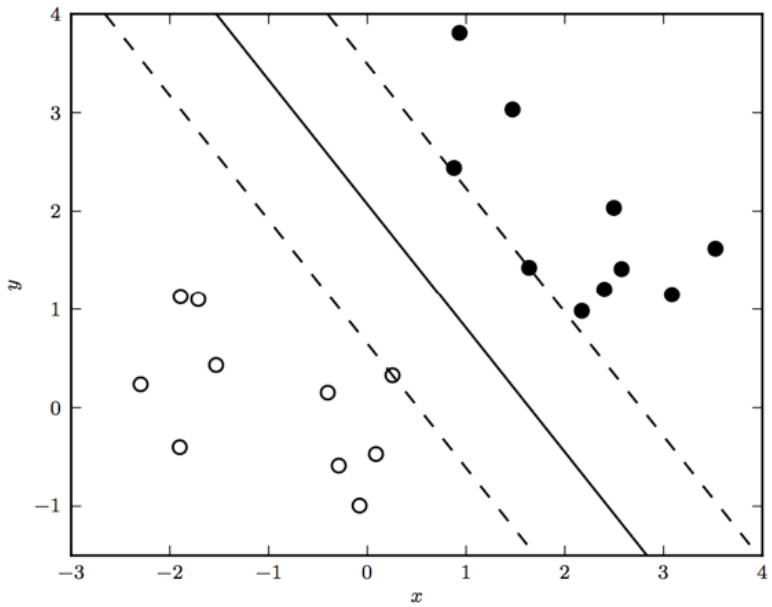
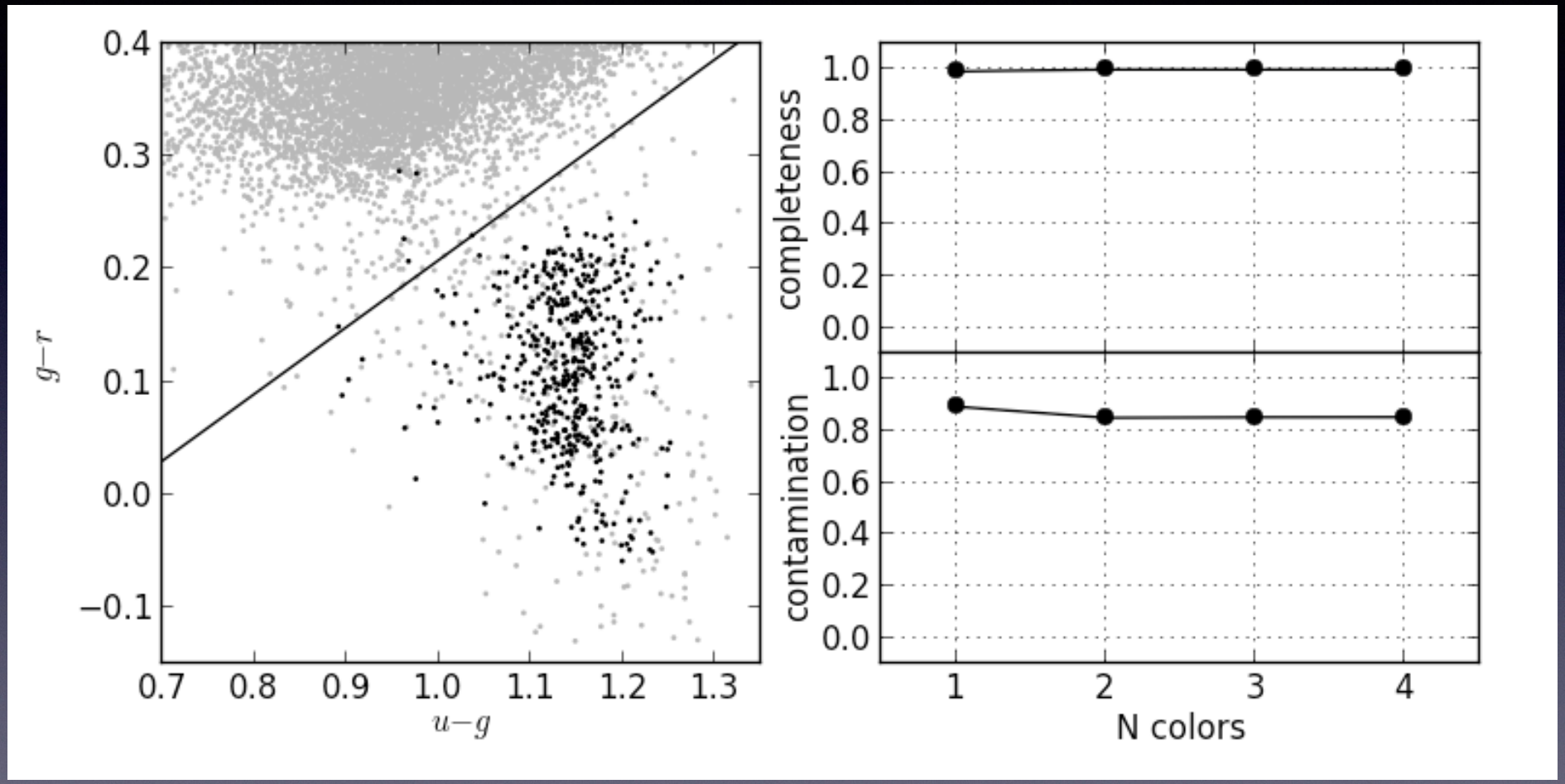


Figure 9.9.: Illustration of SVM. The region between the dashed lines is the *margin*, and the points which the dashed lines touch are called the *support vectors*.

Left: the idea behind the Support Vector Machine classifier: a collection of hyperplanes which maximize the class separation

SVM classification of RR Lyrae

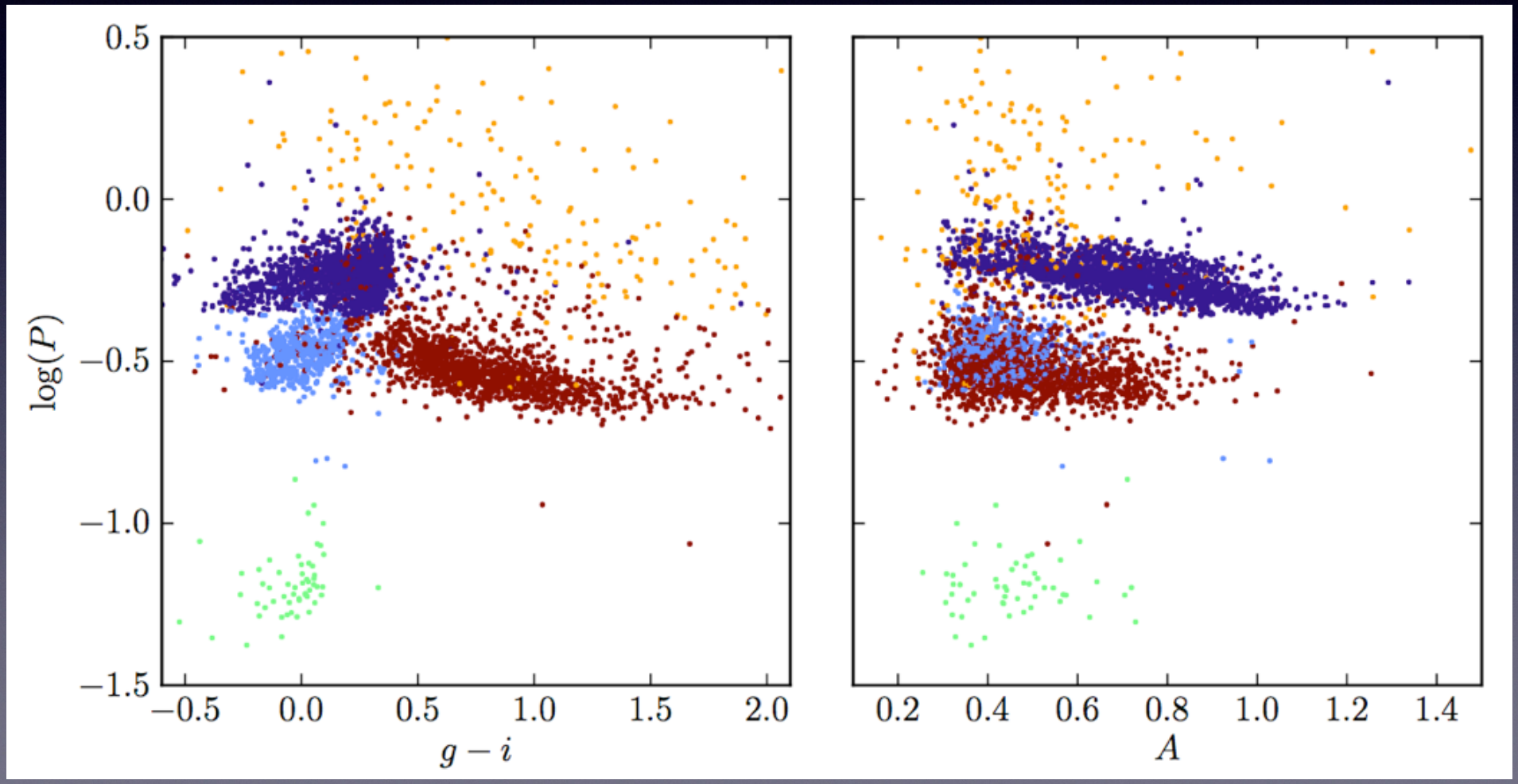
- make this plot by running (it takes a few minutes!)
  %run fig_rrlyrae_svm.py



SVM is very robust to outliers; note the very high completeness (at the expense of high contamination)

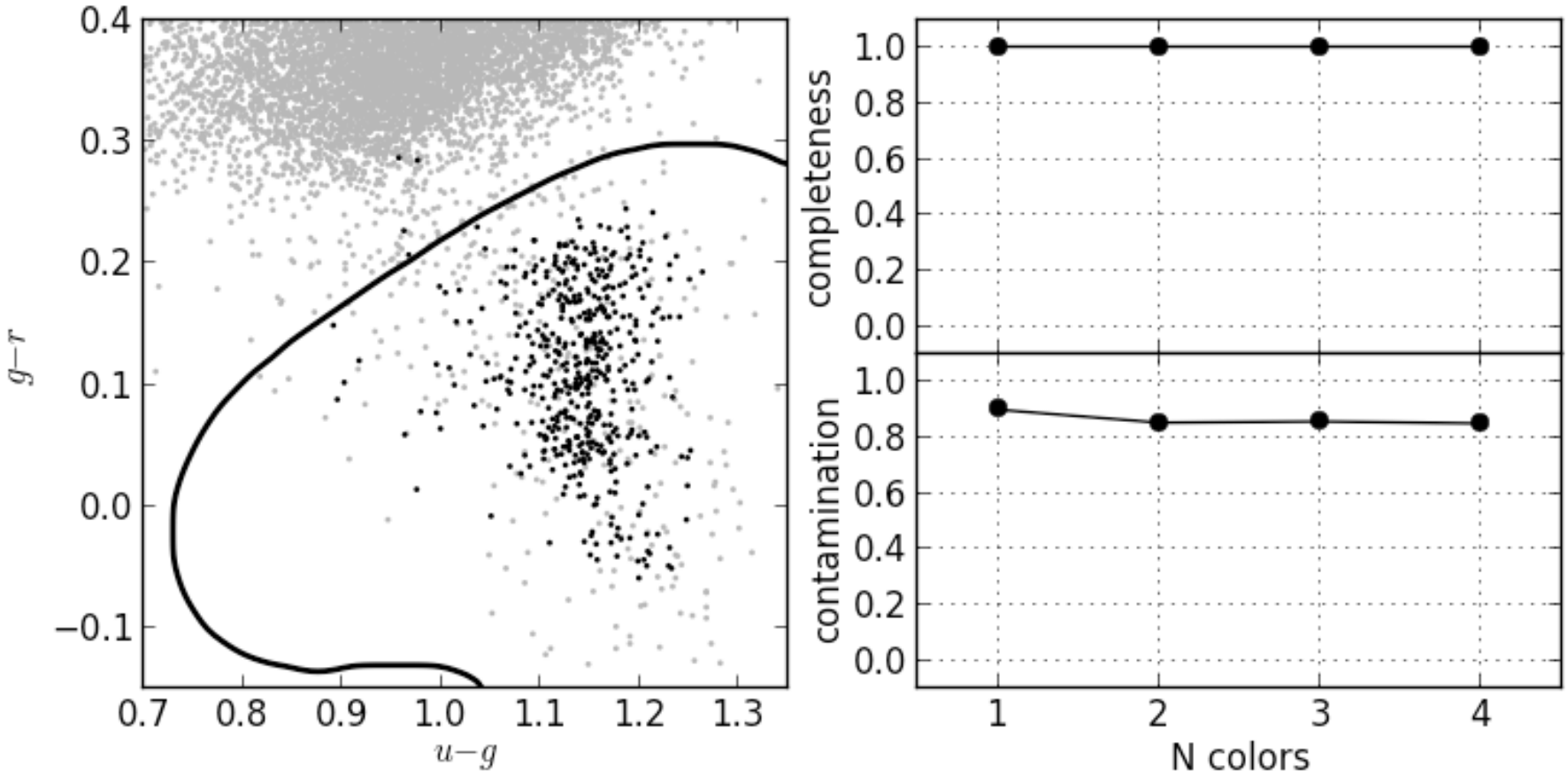
SVM classification of variable stars from Palaversa+ 2013

- Based on 7 attributes (4 SDSS-2MASS colors and 3 light curve parameters: period, amplitude, skewness) and 5 input classes (also in astroML - you can experiment later...)



Kernel SVM classification of RR Lyrae

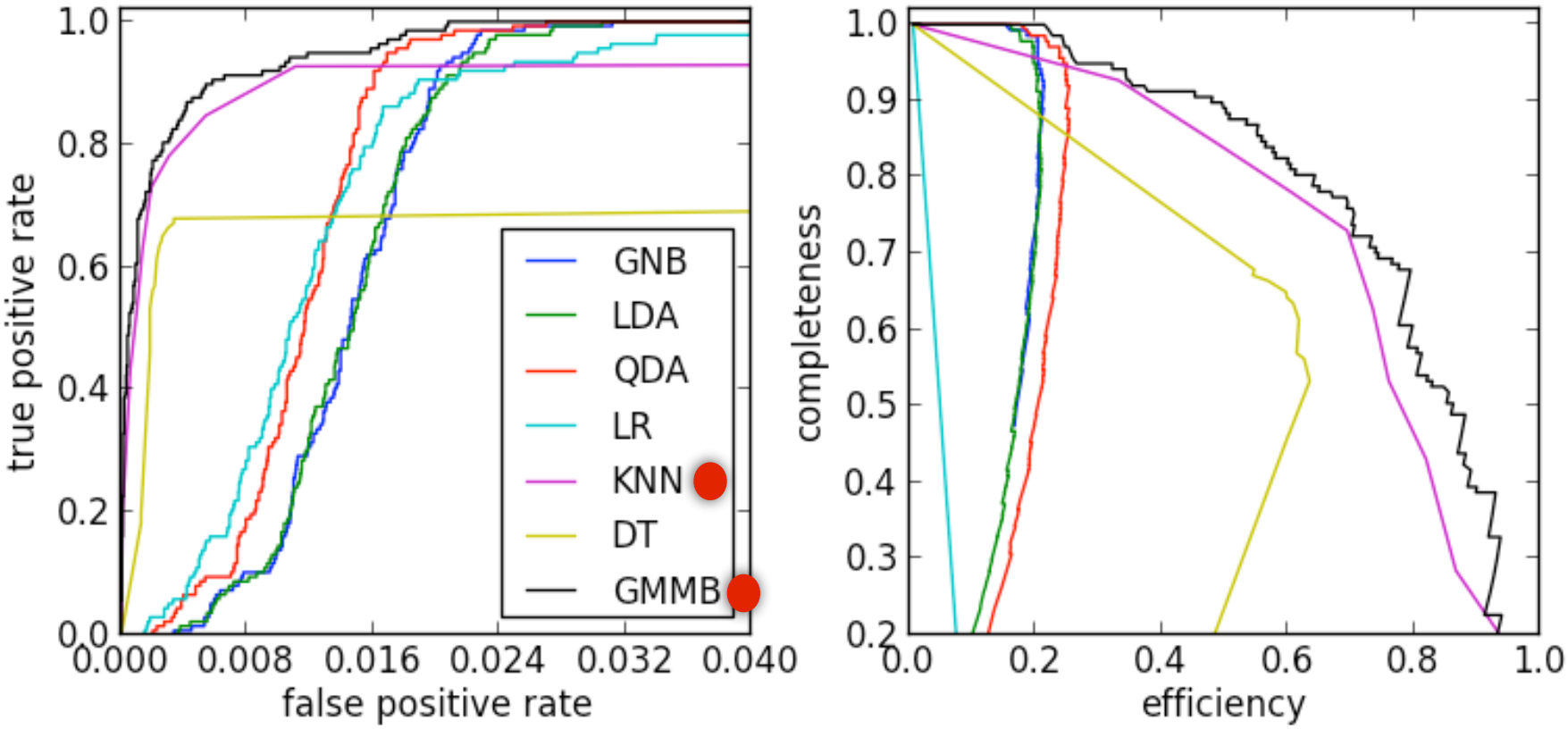
- make this plot by running (it takes a few minutes!)
  %run fig_rrlyrae_kernelsvm.py



The SVM class decision boundaries do not have to be linear (straight lines and hyperplanes): can use kernel tricks

Comparison of classification methods with ROC curves

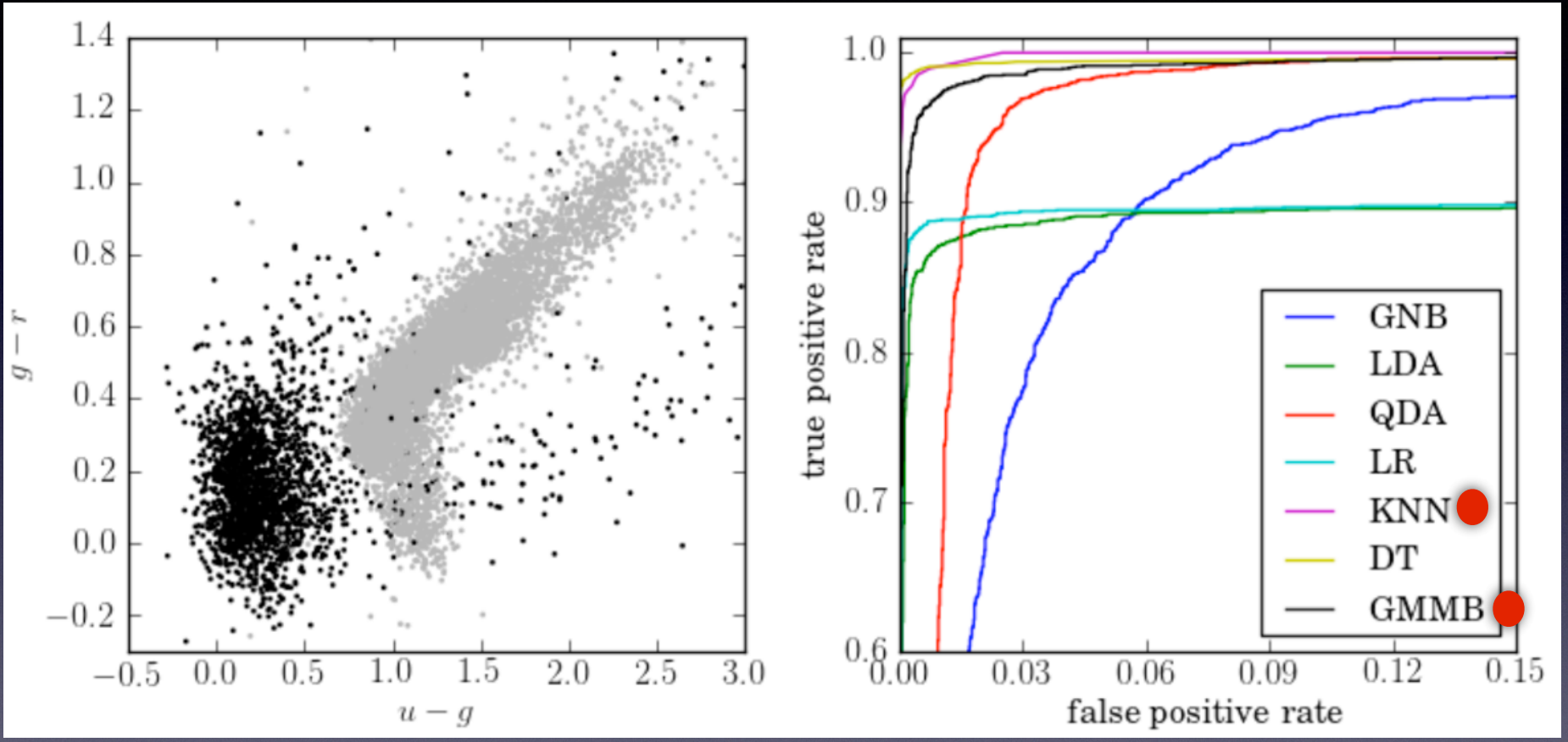
- make this plot by running (it takes a few minutes!)
  %run fig_ROC_curve.py



For each method, completeness and contamination can be optimized using ROC curves, and methods can be compared

Comparison of classification methods with ROC curves

- an example of photometric star (gray) vs. quasar (black) separation using SDSS photometry (UV excess method)



Which method would you choose? Why? Would it fail for a different sample? There are many more methods in astroML, and even many many more in the literature...